

Snort 개념 정리

Snort란 무엇인가?

Snort는 실시간 트래픽 분석 및 패킷 로깅이 가능한 오픈 소스 네트워크 침입 탐지 및 방지 시스템(NIDS/IPS)입니다.



네트워크의 '국경 검문소'처럼, 모든 패킷을 검사하여 악성 행위나 위협을 찾아내는 '탐정' 역할을 수행합니다.

유연한 '룰(Rule)' 기반 언어를 사용하여, 알려진 위협은 물론 새로운 위협 패턴까지 정의하고 실시간으로 대응할 수 있습니다.

Snort

4단계 패킷 프로세스

Snort는 4단계의 과정을 통해 보이지 않는 위협을 찾아냅니다.

4단계 패킷 프로세스



1. 도착 및 캡처

DAQ(데이터 수집 라이브러리)가 네트워크 인터페이스(NIC)에서 원시 패킷을 '낙아칩니다'.



2. 지능형 재조립

전처리기(Preprocessors)가 조각난 패킷과 대화를 '완전한 모습'으로 재구성합니다.
(Evasion 방지)



3. 탐지 및 비교

탐지 엔진(Detection Engine)이 재조립된 데이터를 '룰셋(Ruleset)'과 비교하여 악성 패턴을 검색합니다.



4. 최종 판결 및 조치

룰과 일치할 경우, 출력 플러그인이 '경고(Alert)', '기록(Log)', '차단(Drop)' 등의 조치를 수행합니다.

1단계 상세: DAQ (Data Acquisition)란?



추상화 계층 (Abstraction)

DAQ는 Snort가 '어떻게' 패킷을 가져오는지 신경 쓰지 않게 해주는 '번역기'입니다. 운영 체제(OS)나 네트워크 카드(NIC)에 상관없이 Snort는 DAQ에게 "다음 패킷 줘"라는 동일한 요청만 보냅니다.



플러그인 구조 (Plugin)

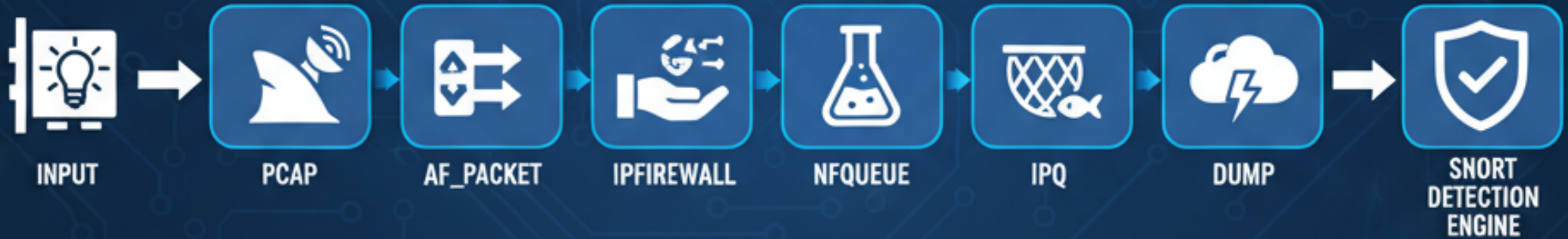
DAQ는 '모듈' 또는 '플러그인' 형태로 작동합니다. 사용자는 Snort 실행 시점에 `--daq` 옵션을 사용하여, 현재 환경에 가장 최적화된 캡처 모듈을 선택하여 사용할 수 있습니다.



Snort와의 분리 (Separation)

Snort 2.9 버전부터, '데이터를 잡아오는(캡처) 부분'이 'DAQ'라는 별도의 부품(모듈)으로 떨어져 나왔습니다.

SNORT DAQ MODULES



유연성 및 이식성

Snort 코드는 그대로 두고 DAQ 모듈만 교체하면, Linux, Windows, FreeBSD 등 어떤 OS에서도 작동합니다.

또한, 특수 하드웨어 캡처 카드(예: Napatech) 제조사가 Snort 코드 변경 없이 자체 고성능 DAQ 모듈을 제공할 수 있습니다.

고성능 캡처

고속 네트워크(10G+) 환경에서는 모든 패킷을 놓치지 않는 것이 중요합니다. `pcap`은 표준이지만, 패킷 손실이 발생할 수 있습니다.

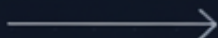
`AFPacket` (Linux)이나 `PF\_RING` 같은 고성능 DAQ 모듈은 커널 단에서 패킷을 직접 처리(Zero-Copy)하여 성능을 극대화합니다.

1단계 활용: 스니퍼 & 로거 모드

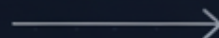
Snort의 핵심은 NIDS(3, 4단계)지만, 1단계(캡처)만으로도 강력한 도구로 사용할 수 있습니다.



스니퍼 모드



패킷 로거 모드



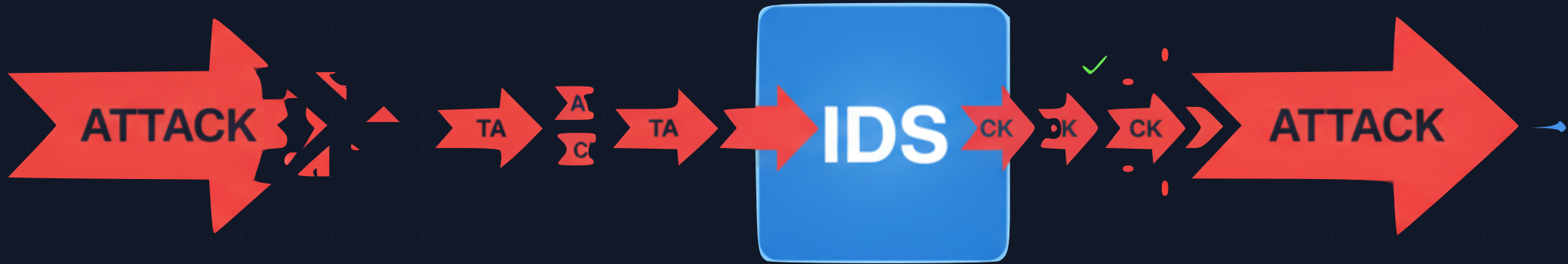
로그 파일 재생

- 스니퍼 모드: 패킷을 실시간으로 캡처하여 화면에 표시합니다. (네트워크 디버깅용) `sudo snort -vde -i eth0`
- 패킷 로거 모드: 캡처한 모든 패킷을 디스크에 로그 파일로 저장합니다. (사후 분석 및 포렌식용) `sudo snort -l ./log -b -i eth0`
- 로그 파일 재생: -r 옵션으로 저장된 pcap 파일을 다시 읽어들이어 NIDS 모드 등으로 분석할 수 있습니다. `snort -r packet.log -c snort.conf`

2단계: 지능형 재조립

공격자가 숨는 방법을 파헤치는 Snort의 핵심 기술입니다.

문제점: 탐지 시스템(IDS) 우회



공격자의 전술: 조각화

공격자는 악성 코드(예: 'ATTACK')를 'AT', 'TA', 'CK'처럼 의미 없는 조각으로 쪼개서 보냅니다.

대부분의 단순한 IDS는 이 조각들만 보고는 위협을 인지하지 못하고 '정상'으로 통과시킵니다.

하지만 이 조각들은 최종 목적지(서버)에 도착하여 'ATTACK'이라는 원래의 악성 코드로 재조립되어 공격에 성공합니다.

해결책: 전처리기 (Preprocessors)

Snort는 탐지 엔진이 '완전한 그림'을 볼 수 있도록 전처리가 먼저 패킷을 재조립합니다.



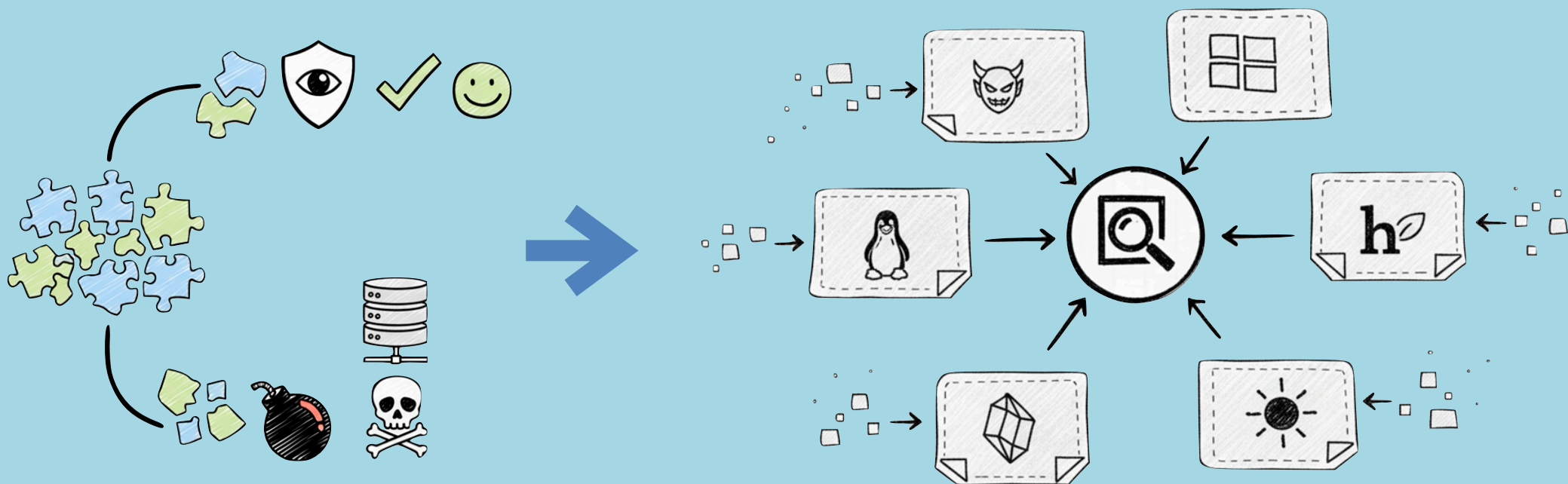
-  **Frag3 (조각모음 3):** IP 단편화(Fragmentation)를 담당합니다. 흩어진 IP 조각들을 모아 하나의 완전한 IP 패킷으로 재조립합니다.
-  **Stream5 (스트림 5):** TCP 세션 재조립을 수행합니다. 여러 패킷에 걸쳐 나뉘어 전송되는 TCP 대화의 전체 흐름(Stream)을 재구성합니다.
-  **HTTP Inspect 등:** HTTP 트래픽의 난독화(예: %2f -> /)를 풀거나, 다른 프로토콜(SMTP, DNS)을 정규화하여 탐지 엔진이 속지 않도록 준비합니다.

타겟 기반 재조립

만약 Snort가 패킷을 재조립하는 방식(A)과 실제 타겟 서버(예: Windows)가 재조립하는 방식(B)이 다르다면?

공격자는 이 차이를 이용해, Snort에게는 '정상(A)'으로 보이지만 서버에게는 '악성(B)'으로 조립되는 특수 조각을 보낼 수 있습니다.

(Ptacek & Newsham 우회 공격)



Snort의 해법: Snort는 '타겟 기반(Target-Based)' 정책을 사용합니다. "저 서버는 Windows야"라고 알려주면, Snort도 정확히 Windows 서버와 동일한 방식으로 패킷을 재조립합니다.

3단계: 탐지 및 룰 적용

재조립된 데이터를 Snort의 '두뇌'로 검사합니다.

Snort의 '두뇌': 탐지 엔진 (Detection Engine)

1. 탐지 엔진 (Detection Engine)

Snort의 핵심 '두뇌'입니다. 전처리가 완벽하게 재조립한 데이터를 룰셋(규칙 집합)과 고속으로 비교하여 일치하는 패턴을 찾아냅니다.

2. 룰셋 (Ruleset)

수천 개의 '수배 전단지' 집합입니다. 알려진 악성 코드 시그니처, 비정상 프로토콜 행위, 정책 위반 등을 정의한 규칙들로 이루어져 있습니다.



룰의 기본 구조: 헤더와 옵션

모든 Snort 룰은 '룰 헤더' (Rule Header)와 '룰 옵션' (Rule Options)이라는 두 가지 논리적 섹션으로 구성됩니다.

룰 헤더 (Rule Header)

'누가', '어디서', '어디로', '무엇을'에 해당하는 패킷의 기본 정보를 정의합니다. (Action, Protocol, IP, Port...)

룰 옵션 (Rule Options)

괄호 () 안에 위치하며, '왜' 이 패킷이 악성인지를 정의하는 세부 탐지 조건입니다. (msg, content, flow, sid...)

룰 헤더

```
alert tcp any any ->  
$HOME_NET 21
```

룰 옵션

```
(content:"USER root";  
msg:"FTP root login  
attempt";)
```

TCP 통신 프로토콜
일 경우 Alert. 즉,
경고를 발생시켜야
합니다.

any
출발지(Source)
에 관계없이

alert tcp any any ->

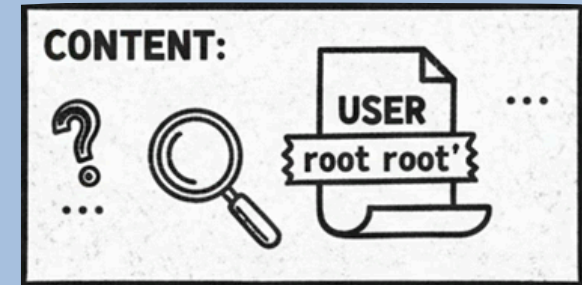


\$HOME_NET 21



우리 내부망의 21번 포트
(FTP 서버)로
유입되는 데이터 중에서

(content:"USER root";
msg:"FTP root login
attempt";)



데이터에 'user root'
문자열이 포함될 경우,
즉시 경고를 발생시킵니다.

4단계: 최종 판결 및 조치

탐지된 위협에 대해 Snort가 동작을 수행합니다.

5가지 주요 룰 액션 (Rule Actions)



alert

경고를 생성하고 해당 패킷을 로깅합니다.
(NIDS 모드의 기본)



log

경고 없이 패킷만 로깅합니다.
(트래픽 수집용)



pass

패킷을 무시하고 통과시킵니다.
(오탐 제외 및 화이트리스트링)



drop

패킷을 차단(Drop)하고 로깅합니다.
(IPS 모드 전용)



reject

패킷을 차단, 로깅하고 TCP Reset 또는 ICMP Unreachable 응답을 전송합니다.